

API REFERENCE — Modulo Booking

Backend Gestionale B&B Cosenza Versione API v1 · Documento aggiornato al 20 settembre 2026 · Copertura: Step A → D

Come usare questo documento

Serve a tre scopi, e ogni sezione è pensata per uno di essi.

Per il collaudo manuale: la sezione *Endpoint* descrive ogni rotta con i suoi codici di risposta e un esempio curl pronto da incollare. La *Checklist di verifica* in fondo è la sequenza da seguire a sviluppo concluso.

Per lo sviluppo frontend: la sezione *Schemi* elenca ogni campo con tipo, vincoli e obbligatorietà. La sezione *Meccanismi trasversali* spiega cosa il client deve inviare oltre ai dati veri e propri (token del preventivo, captcha, honeypot) e come interpretare gli errori.

Per chi riprende lo sviluppo: la *Logica* di ciascun endpoint dice cosa accade dietro la chiamata e quali test la coprono.

Questo documento viene aggiornato a ogni modifica del modulo. Il *Changelog* in fondo traccia cosa è cambiato e quando.

1. Convenzioni generali

1.1 Base URL e versioning

```
http://localhost:8000/api/v1
```

Tutte le rotte del modulo sono sotto /api/v1/bookings.

1.2 Formato delle date

Campo	Formato	Esempio
Date di soggiorno (check_in , check_out)	YYYY-MM-DD (solo data)	2026-12-01
Istanti (hold_expires_at , confirmed_at , ...)	ISO 8601 con fuso, in UTC	2026-12-01T14:30:00Z

⚠ Nota per il frontend. Le date di soggiorno **non** sono timestamp. check_out è il giorno di partenza ed è **escluso** dal soggiorno: 2026-12-01 → 2026-12-03 sono **due notti**. Prima dello Step A questi campi erano timestamp completi: se esiste codice Angular che li tratta come tali, va adeguato.

1.3 Importi

Tutti gli importi sono **stringhe decimali con due cifre** ("200.00"), non numeri in virgola mobile. Serializzarli come float in JavaScript introduce errori di arrotondamento sui totali.

La valuta è in currency (attualmente sempre "EUR").

1.4 Formato degli errori

Ogni errore ha la stessa forma:

```
{
  "message": "Testo leggibile, già in italiano, mostrabile all'utente"
}
```

Gli errori di validazione hanno un campo aggiuntivo:

```
{
  "message": "La password deve contenere almeno una maiuscola",
  "details": [
    { "type": "value_error", "loc": ["body", "password"], "msg": "..."}
  ]
}
```

`details` serve al frontend per evidenziare il campo in errore (`loc` contiene il percorso). **Il valore rifiutato non viene mai restituito**: `details` non contiene la chiave `input`, per evitare che una password o un dato personale torni indietro nella risposta.

1.5 Codici di stato e loro significato

Codice	Quando	Cosa deve fare il client
200	Operazione riuscita	—
201	Prenotazione creata	—
400	Token di conferma o cancellazione non valido	Invitare a richiedere un nuovo link
401	Sessione assente o scaduta	Reindirizzare al login
403	Ruolo insufficiente	Nascondere la funzione
404	Risorsa inesistente, o non di proprietà dell'utente	Messaggio generico
409	Conflitto: slot occupato, transizione non ammessa, già confermata	Mostrare il messaggio e proporre una nuova ricerca
410	Blocco temporaneo scaduto	Invitare a ripetere la prenotazione
422	Dati non validi, preventivo scaduto, captcha fallito	Evidenziare il campo tramite <code>details</code>
429	Troppe richieste	Attendere i secondi indicati in <code>Retry-After</code>
402	Pagamento richiesto o fallito (<i>dallo Step G</i>)	—

1.6 Mappa eccezione → codice

Il backend non espone mai stack trace. Ogni eccezione di dominio deriva da `AppException` e porta con sé il proprio codice HTTP; un **unico handler** globale le traduce in risposta.

Eccezione	Codice	Messaggio predefinito
<code>EntityNotFound</code>	404	Entity not found
<code>EntityAlreadyExists</code>	409	Entity already exists
<code>EntityInUse</code>	409	L'elemento è utilizzato da altri dati e non può essere eliminato
<code>RoomNotAvailable</code>	409	Una o più camere non sono disponibili per le date richieste
<code>InvalidBookingStatusTransition</code>	409	L'operazione non è consentita nello stato attuale
<code>BookingNotCancellable</code>	409	La prenotazione non può più essere cancellata
<code>BookingHoldExpired</code>	410	Il tempo per completare la prenotazione è scaduto
<code>InvalidBookingToken</code>	400	Il link utilizzato non è valido o è già stato usato
<code>InvalidDateRange</code>	422	L'intervallo di date indicato non è valido
<code>InvalidGuestCount</code>	422	Il numero di ospiti non è compatibile con le camere
<code>InvalidQuoteToken</code>	422	Il preventivo non è più valido
<code>CaptchaValidationFailed</code>	422	Verifica anti-bot non superata
<code>RateLimitExceeded</code>	429	Troppe richieste: riprova più tardi
<code>InvalidFileType</code> / <code>InvalidFileSize</code>	422	—
<code>StaleDataError</code> (SQLAlchemy)	409	Il dato è stato modificato da un'altra operazione

2. Autenticazione e autorizzazione

2.1 Come funziona

Il progetto usa **JWT veicolati da cookie** `HttpOnly`, non header `Authorization`. Il token non è leggibile da JavaScript, il che elimina il furto per Cross-Site Scripting.

Login su `POST /api/v1/auth/token` con form data (`username` = email, `password`). La risposta imposta due cookie:

Cookie	Durata	Attributi
<code>access_token</code>	breve (<code>access_token_expire_minutes</code>)	<code>HttpOnly</code> , <code>SameSite=lax</code> , <code>Path=</code>
<code>refresh_token</code>	lunga (<code>refresh_token_expire_minutes</code>)	<code>HttpOnly</code> , <code>SameSite=lax</code> , <code>Path=</code>

⚠ **Frontend:** tutte le chiamate autenticate devono usare `withCredentials: true` (Angular `HttpClient`) o `credentials: 'include'` (fetch). Senza, il cookie non viene inviato e la risposta è `401`.

⚠ **Produzione:** i cookie hanno attualmente `secure=False`. Va portato a `True` con HTTPS attivo. (Voce di backlog, non ancora risolta.)

2.2 Livelli di accesso

Livello	Dipendenza	Comportamento
Pubblico	nessuna	Chiunque, anche non registrato
Autenticato	<code>Depends(get_current_user)</code>	Cookie <code>access_token</code> valido; altrimenti <code>401</code>
Amministrativo	<code>Depends(is_admin_user)</code>	Utente con <code>role = admin</code> ; altrimenti <code>403</code>

`is_admin_user` è un `RoleChecker(allowed_roles=[UserRole.admin])` .

2.3 Cosa ha aggiunto il modulo Booking

Nessuna modifica al meccanismo di autenticazione esistente. Il modulo introduce però un secondo sistema di credenziali, **indipendente e non sostitutivo**:

- **Token monouso di prenotazione** — consentono a un ospite *non registrato* di confermare o cancellare la propria prenotazione senza avere un account. Non sono sessioni: valgono per una sola operazione su una sola prenotazione.

I due sistemi non si incrociano: un token di prenotazione non autentica l'utente, e una sessione valida non sostituisce il token nel link ricevuto via email.

3. Meccanismi trasversali introdotti

Questa sezione descrive tutto ciò che è stato aggiunto al progetto a supporto del modulo Booking.

3.1 Rate limiting

File: `src/security/rate_limiter.py`

Finestra scorrevole: per ogni chiave si conservano gli istanti delle richieste recenti e si scartano quelle uscite dalla finestra. A differenza del conteggio a finestra fissa, impedisce il raddoppio del limite a cavallo di due intervalli.

Limiti attivi (configurabili da `.env`):

Ambito	Limite	Finestra	Chiave
<code>GET /availability</code>	20	1 minuto	IP
<code>POST /quote</code>	20	1 minuto	IP
<code>POST /</code> (creazione)	5	1 ora	IP
<code>POST /</code> (creazione)	3	24 ore	email
<code>POST /confirm</code> , <code>/cancel</code>	10	1 ora	IP
<code>POST /lookup</code>	10	1 ora	IP

I limiti per IP ed email sono complementari: un attaccante con molti indirizzi IP resta vincolato sul numero di prenotazioni riconducibili alla stessa email, e viceversa.

Risposta al superamento: `429` con header `Retry-After` in secondi.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 42
```

```
{"message": "Troppe richieste: riprova più tardi"}
```

Due limiti noti, da tenere presenti:

1. Il **contatore è per processo**. Con `uvicorn --workers 4` ogni worker ha il proprio, quindi il limite effettivo è il quadruplo. Su un singolo worker il conteggio è esatto. L'interfaccia `RateLimitBackend` consente di sostituire lo storage con Redis senza toccare i router.
2. **X-Forwarded-For è ignorato per impostazione predefinita** (`trusted_proxy_count = 0`). Quell'header lo scrive il client: fidarsene senza un proxy che lo riscriva permetterebbe a chiunque di cambiare IP a ogni richiesta e azzerare il rate limiting. Quando l'app andrà dietro Nginx o Cloudflare, impostare `TRUSTED_PROXY_COUNT=1`.

3.2 Captcha — Cloudflare Turnstile

File: `src/security/captcha.py`

Verificato solo su `POST /api/v1/bookings/` (creazione pubblica). Scelto al posto di reCAPTCHA perché non profila gli utenti e non richiede un consenso ai cookie aggiuntivo.

- Disattivato per impostazione predefinita (`CAPTCHA_ENABLED=false`): in sviluppo non serve configurare nulla.
- Il frontend inserisce il widget Turnstile e invia il token risultante nel campo `captcha_token`.
- Token assente o rifiutato → `422`.

Comportamento in caso di disservizio: se Cloudflare non risponde o va in timeout (5 secondi), la richiesta **viene lasciata passare** e l'evento registrato come warning. Bloccare tutte le prenotazioni durante un disservizio di terze parti costerebbe incassi reali; le poche richieste automatiche che passassero creerebbero prenotazioni non confermate, che scadono da sole in quindici minuti.

3.3 Honeypot

Il campo `website` di `GuestBookingCreateSchema` è una trappola: va reso **invisibile via CSS** nel form e mai compilato da un utente reale. Se arriva valorizzato, la richiesta è automatizzata e viene respinta con `422`.

Costa nulla e intercetta i bot che compilano ogni campo del form.

3.4 Preventivo firmato (`quote_token`)

File: `src/security/quote_token.py`

Il client non invia mai un prezzo. Il flusso pubblico è a due passi per costruzione:

1. `POST /quote` calcola il totale e lo restituisce racchiuso in un JWT firmato, valido 15 minuti;
2. `POST /` accetta esclusivamente quel token.

Il token contiene camere, date, numero di ospiti, opzione di pagamento e totale. Protegge da due cose: la **manomissione del prezzo**, perché la firma lo rende immodificabile, e lo **spostamento della selezione**, perché non si possono sostituire le camere dopo il calcolo.

È firmato con lo stesso segreto dell'autenticazione ma con un claim `type` diverso: un access token non può essere speso come preventivo, né viceversa.

Difesa in profondità: anche con un token valido, il server **ricalcola** il totale alla creazione e rifiuta con `422` se diverge. La firma dice "questo prezzo l'ho emesso io", non "questo prezzo è ancora corretto".

3.5 Token monouso di prenotazione

File: `src/security/booking_tokens.py`

Generati con `secrets.token_urlsafe(32)` — 256 bit di entropia. Nel database finisce **solo l'hash SHA-256**, mai il valore in chiaro: una compromissione del database non consente di confermare o cancellare prenotazioni altrui.

Scopo	Uso
<code>CONFIRM_EMAIL</code>	Conferma della prenotazione (solo <code>PAY_ON_ARRIVAL</code>)
<code>MANAGE</code>	Gestione della prenotazione
<code>CANCEL</code>	Cancellazione

Sono **monouso**: al primo utilizzo `used_at` viene valorizzato e i tentativi successivi respinti.

⚠ **Il token viaggia nel body, mai in query string.** Un token nell'URL finisce negli access log, nell'header `Referer` e nella cronologia del browser. Il link nell'email punta al **frontend** (`{frontend_base_url}/booking/confirm?token=...`), e la SPA lo inoltra al backend con una `POST`.

3.6 Esposizione temporanea del token in sviluppo

Finché l'invio email non esiste (Step F), `POST /` restituisce il token di conferma nel campo `confirmation_token`, **ma solo quando** `EMAIL_ENABLED=false`. Attivando l'invio email il campo torna `null` automaticamente.

C'è un test che lo verifica, così l'affordance si spegne da sé senza che nessuno debba ricordarsene.

3.7 Gestione centralizzata degli errori

File: `src/exception/exception_handler.py`

Un **unico handler** registrato su `AppException` copre tutte le eccezioni di dominio, presenti e future: Starlette risolve percorrendo l'MRO dell'eccezione. Aggiungere un nuovo errore richiede una sola classe, non anche un handler.

Correzione importante introdotta allo Step B: gli errori di validazione producevano un `500` invece di un `422`, perché Pydantic v2 inserisce l'**oggetto eccezione vivo** nella chiave `ctx` e `json.dumps` non sa serializzarlo. Ora il contenuto di `ctx` viene convertito in stringa e le chiavi `input` e `url` rimosse.

4. Endpoint

4.1 Prospetto

#	Metodo	Rotta	Accesso	Rate limit	Captcha
1	GET	/api/v1/bookings/availability	Pubblico	20/min IP	no
2	POST	/api/v1/bookings/quote	Pubblico	20/min IP	no
3	POST	/api/v1/bookings/	Pubblico	5/h IP + 3/g email	sì
4	POST	/api/v1/bookings/confirm	Pubblico	10/h IP	no
5	POST	/api/v1/bookings/cancel	Pubblico	10/h IP	no
6	POST	/api/v1/bookings/lookup	Pubblico	10/h IP	no
7	GET	/api/v1/bookings/me	Autenticato	—	no
8	POST	/api/v1/bookings/me	Autenticato	—	no
9	POST	/api/v1/bookings/me/{booking_id}/cancel	Autenticato	—	no

1 • GET /api/v1/bookings/availability

Camere disponibili nell'intervallo richiesto.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Query	<code>check_in</code> (date, obbligatorio) · <code>check_out</code> (date, obbligatorio) · <code>guest_count</code> (int ≥ 1, default 1)
Risposta	<code>AvailabilityResponseSchema</code>

Codici: `200` · `422` date non valide · `429` limite superato

Logica. Recupera le camere abilitate e sottrae quelle occupate nell'intervallo. Una camera risulta occupata quando esiste una riga attiva con date sovrapposte e la prenotazione è in uno stato occupante — per gli stati temporanei, solo se il blocco non è ancora scaduto.

Non filtra per capienza, di proposito: nasconderebbe la possibilità di prenotare due camere doppie per quattro persone, che in un B&B è il caso normale delle famiglie. Restituisce invece tutte le camere libere con il flag `fits_all_guests`, più le combinazioni utili.

Le tre modalità di prenotazione sono servite da questa singola chiamata:

Modalità	Come ottenerla dal frontend
Camera singola sufficiente	Filtrare <code>rooms</code> su <code>fits_all_guests == true</code>
Scelta manuale	Presentare <code>rooms</code> per intero — la somma delle capienze è verificata al preventivo
Combinazioni suggerite	Usare <code>suggested_combinations</code>

Le combinazioni sono minimali: se togliendo una camera gli ospiti ci starebbero ancora, la proposta non viene restituita. Con una doppia, un'altra doppia e una quadrupla, per 4 persone si ottengono due proposte — la quadrupla da sola, oppure le due doppie — e non le varianti ridondanti. Ordinate per numero di camere, poi prezzo, poi capienza sprecata. Massimo 10 proposte.

Test: `test_flusso_completo_availability_quote_create_confirm`, `test_availability_marca_le_camere_che_bastano_da_sole`, `test_availability_propone_combinazioni`, `test_availability_date_incoerenti_rifiutate`, `test_rate_limit_con_retry_after`, più l'intero file `test_availability_combinations.py` (11 test sulla logica pura).

```
curl -i "http://localhost:8000/api/v1/bookings/availability?check_in=2026-12-01&check_out=2026-12-03&g
```

2 • `POST /api/v1/bookings/quote`

Preventivo firmato per una selezione di camere.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Body	<code>BookingQuoteRequestSchema</code>
Risposta	<code>BookingQuoteResponseSchema</code>

Codici: `200` • `404` camera inesistente • `409` camera occupata o disabilitata • `422` date non valide, ospiti oltre la capienza • `429`

Logica. Carica le camere, verifica che esistano e siano abilitate, controlla che la somma delle capienze copra gli ospiti, verifica la disponibilità, calcola il prezzo ed emette il token firmato.

È **di sola lettura**: non blocca nulla e non crea righe. La disponibilità viene verificata per dare all'utente un errore immediato invece di farglielo scoprire dopo aver compilato i propri dati, ma **la garanzia resta al momento della prenotazione**: fra il preventivo e la conferma qualcun altro può sempre arrivare prima.

Lo sconto per pagamento online (10%) è applicato qui e incluso nel token.

Test: `test_flusso_completo...`, `test_preventivo_su_camera_inesistente`, e l'intero `test_pricing_service.py` (18 test su calcoli, arrotondamento e token).

```
curl -i -X POST http://localhost:8000/api/v1/bookings/quote \
-H "Content-Type: application/json" \
-d '{"check_in":"2026-12-01","check_out":"2026-12-03","guest_count":2,
  "room_ids":["<UUID_CAMERA>"],"payment_option":"PAY_ON_ARRIVAL"}'
```

3 • `POST /api/v1/bookings/`

Crea una prenotazione come ospite non registrato.

Accesso	Pubblico
Rate limit	5/ora per IP e 3/giorno per email
Captcha	Sì (se <code>CAPTCHA_ENABLED=true</code>)
Body	<code>GuestBookingCreateSchema</code>
Risposta	<code>BookingCreatedSchema</code> — 201

Codici: 201 · 404 camera inesistente · 409 slot occupato · 410 — · 422 preventivo scaduto o manomesso, honeypot, condizioni non accettate, captcha fallito · 429

Logica — l'operazione più delicata del modulo. Tutto dentro una sola transazione:

1. verifica del `quote_token` (firma e scadenza);
2. caricamento camere, controllo esistenza e abilitazione;
3. controllo capienza totale $\geq$ ospiti;
4. **ricalcolo del prezzo** e confronto con il token $\rightarrow$ 422 se diverge;
5. controllo del numero di prenotazioni in attesa per quella email;
6. **lock pessimistico** sulle camere, ordinato per id (nessun deadlock possibile);
7. **just-in-time expiration**: libera gli slot il cui blocco è scaduto;
8. verifica disponibilità $\rightarrow$ 409 con l'elenco delle camere non più libere;
9. generazione del codice prenotazione univoco;
10. creazione della prenotazione e delle righe camera, con prezzo congelato;
11. `flush` $\rightarrow$ **qui scatta l'exclusion constraint del database**: se due richieste simultanee arrivano insieme, una sola passa;
12. riga di storico e token di conferma.

Stato iniziale:

Opzione di pagamento	Stato	Token di conferma
<code>PAY_ON_ARRIVAL</code>	<code>PENDING_CONFIRMATION</code>	sì
<code>PAY_NOW</code>	<code>PENDING_PAYMENT</code>	no — il pagamento è la verifica d'identità

Lo slot resta bloccato **15 minuti** (`hold_expires_at`). Scaduto quel termine senza conferma, torna prenotabile.

Test: `test_flusso_completo...` , `test_slot_occupato_risponde_409` , `test_honeypot_compilato_rifiutato` , `test_condizioni_non_accettate_rifiutate` , `test_token_di_conferma_non_esposto_con_email_attiva` , più i 14 test di `test_booking_service.py` fra cui `test_due_prenotazioni_concorrenti_una_sola_vince` , ripetuto 10 volte.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/ \
-H "Content-Type: application/json" \
-d '{"quote_token":"<TOKEN_DAL_PREVENTIVO>",
  "guest":{"firstname":"Mario","lastname":"Rossi",
    "email":"mario@example.com","phone_number":"3331234567"},
  "accept_terms":true}'
```

4 • POST /api/v1/bookings/confirm

Conferma tramite il token ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	BookingConfirmSchema
Risposta	BookingPublicSchema

Codici: 200 • 400 token inesistente o già speso • 409 già confermata • 410 blocco scaduto • 429

Logica. Cerca il token per hash, verifica che sia del tipo corretto e non usato, controlla che né il token né il blocco siano scaduti, esegue la transizione a **CONFIRMED**, azzerava **hold_expires_at**, calcola il termine di cancellazione gratuita, marca il token come speso e invalida gli altri della stessa prenotazione.

Doppio clic sul link: se il token è già speso e la prenotazione è **CONFIRMED**, la risposta è 409 con *"La prenotazione è già stata confermata"* — distinto dal generico "link non valido", per non far dubitare l'ospite che la conferma sia andata a buon fine.

Blocco scaduto: 410. Lo stato resta **PENDING_CONFIRMATION** finché non passa lo sweeper (Step F); lo slot è comunque già tornato prenotabile.

Test: `test_flusso_completo_...`, `test_doppia_conferma_risponde_409`, `test_conferma_completa_il_ciclo`, `test_doppia_conferma_segnalata`, `test_conferma_dopo_la_scadenza_rifiutata`.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/confirm \
  -H "Content-Type: application/json" -d '{"token":"<TOKEN_DI_CONFERMA>"}'
```

5 • POST /api/v1/bookings/cancel

Cancellazione tramite link ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	BookingCancelSchema
Risposta	BookingPublicSchema

Codici: 200 • 400 token non valido • 409 non cancellabile • 429

Logica. Accetta token di tipo **CANCEL** o **MANAGE**. Verifica che la cancellazione sia ammessa:

Situazione	Esito
Stato <code>PENDING_*</code>	Consentita
<code>CONFIRMED</code> + <code>PAY_ON_ARRIVAL</code> entro il termine	Consentita, gratuita
<code>CONFIRMED</code> + <code>PAY_ON_ARRIVAL</code> oltre il termine	<code>409</code> — invito a contattare la struttura
<code>CONFIRMED</code> + <code>PAY_NOW</code>	<code>409</code> — non rimborsabile, invito a contattare la struttura
Stato terminale	<code>409</code>

Alla cancellazione lo slot torna **immediatamente** prenotabile.

Test: `test_slot_liberato_dopo_la_cancellazione` .

6 • `POST /api/v1/bookings/lookup`

Consulta una prenotazione con codice ed email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	<code>BookingLookupSchema</code>
Risposta	<code>BookingPublicSchema</code>

Codici: `200` • `404` nessuna corrispondenza • `429`

Logica. Richiede codice e email: il solo codice non basta, per evitare che tentativi a forza bruta espongano dati di altri ospiti. Il confronto sull'email è insensibile alle maiuscole; il codice viene normalizzato (spazi rimossi, maiuscolo).

Il `404` ha un messaggio **generico e identico** in ogni caso di fallimento: non deve lasciar capire se il codice esista e sia soltanto l'email a non corrispondere.

Test: `test_lookup_con_dati_errati_non_rivela_nulla` , `test_lookup_con_codice_ed_email_corretti` .

7 • `GET /api/v1/bookings/me`

Le prenotazioni dell'utente autenticato.

Accesso	Autenticato (cookie <code>access_token</code>)
Query	<code>limit</code> (1–100, default 20) • <code>offset</code> (≥ 0, default 0)
Risposta	<code>List[BookingPublicSchema]</code>

Codici: `200` • `401` sessione assente o scaduta

Ordinate per data di arrivo decrescente.

Test: `test_le_mie_prenotazioni_richiedono_autenticazione` .

8 • POST /api/v1/bookings/me

Crea una prenotazione come utente autenticato.

Accesso	Autenticato
Body	UserBookingCreateSchema
Risposta	BookingCreatedSchema — 201

Codici: 201 • 401 • 404 • 409 • 422

Logica. Identica alla creazione pubblica, con tre differenze: nessun captcha e nessun limite per email (la sessione autenticata è già una barriera contro l'automazione), e l'anagrafica **copiata dal profilo** invece che richiesta.

Lo snapshot anagrafico resta comunque congelato: una modifica successiva all'account non altera la prenotazione.

Test: test_creazione_utente_richiede_autenticazione .

9 • POST /api/v1/bookings/me/{booking_id}/cancel

Cancella una propria prenotazione.

Accesso	Autenticato
Path	booking_id (UUID)
Body	OwnBookingCancelSchema
Risposta	BookingPublicSchema

Codici: 200 • 401 • 404 inesistente o di un altro utente • 409 non cancellabile

Perché 404 e non 403 su una prenotazione altrui: distinguere i due casi confermerebbe l'esistenza dell'identificativo e consentirebbe di sondare le prenotazioni di altri utenti.

5. Schemi

5.1 Richiesta

BOOKINGQUOTEREQUESTSCHEMA

Campo	Tipo	Obbl.	Vincoli
check_in	date	sì	≥ oggi, entro 365 giorni
check_out	date	sì	> check_in , da 1 a 30 notti
guest_count	int	sì	1–100
room_ids	UUID[]	sì	non vuota, senza duplicati, max 5
payment_option	enum	sì	PAY_NOW PAY_ON_ARRIVAL

GUESTDATASchema

Campo	Tipo	Obbl.	Vincoli
firstname	string	sì	2–50 caratteri, senza spazi esterni
lastname	string	sì	2–50 caratteri, senza spazi esterni
email	string	sì	email valida (RFC)
phone_number	string	sì	esattamente 10 cifre numeriche

GUESTBOOKINGCREATESchema

Campo	Tipo	Obbl.	Vincoli
quote_token	string	sì	20–4096 caratteri
guest	GuestDataSchema	sì	—
accept_terms	bool	sì	deve essere true
captcha_token	string	no	max 4096
website	string	no	honeypot: deve restare vuoto

USERBOOKINGCREATESchema

Campo	Tipo	Obbl.
quote_token	string	sì
accept_terms	bool	sì (deve essere true)

BOOKINGCONFIRMSchema · BOOKINGCANCELSCHEMA · OWNBOOKINGCANCELSCHEMA · BOOKINGLOOKUPSCHEMA

Schema	Campi
BookingConfirmSchema	token (string, 20–512, obbligatorio)
BookingCancelSchema	token (obbligatorio) · reason (max 500, opzionale)
OwnBookingCancelSchema	reason (max 500, opzionale) — nessun token: basta la sessione
BookingLookupSchema	code (3–20, normalizzato in maiuscolo) · email

SCHEMI AMMINISTRATIVI (ENDPOINT ALLO STEP E)

Schema	Campi principali
AdminBookingCreateSchema	date, <code>guest_count</code> , <code>room_ids</code> , <code>payment_option</code> , <code>payment_method?</code> , <code>user_id</code> XOR <code>guest</code> , <code>skip_email_confirmation</code> (default <code>true</code>), <code>mark_as_paid</code> (default <code>false</code>), <code>admin_notes?</code> — consente date nel passato
BookingStatusUpdateSchema	<code>new_status</code> , <code>reason?</code> — obbligatoria se <code>new_status</code> = <code>CANCELLED</code>
AdminPaymentRegistrationSchema	<code>payment_method</code> , <code>payment_status</code> , <code>amount?</code>
BookingExtendHoldSchema	<code>minutes</code> (1–120)
BookingSearchFiltersSchema	<code>status[]?</code> , <code>date_from?</code> , <code>date_to?</code> , <code>email?</code> , <code>code?</code> , <code>room_id?</code> , <code>page</code> (default 1), <code>page_size</code> (default 20, max 100)

Nessuno schema di input contiene un campo prezzo. C'è un test che lo verifica.

5.2 Risposta

AVAILABILITYRESPONSESCHEMA

Campo	Tipo	Note
<code>check_in</code> / <code>check_out</code>	date	—
<code>nights</code>	int	notte di soggiorno
<code>guest_count</code>	int	—
<code>rooms</code>	<code>AvailableRoomSchema[]</code>	ordinate per numero camera
<code>suggested_combinations</code>	<code>RoomCombinationSchema[]</code>	max 10, minimali

AVAILABLEROOMSCHEMA

Campo	Tipo	Note
<code>id</code> · <code>name</code> · <code>number</code> · <code>capacity</code>	UUID, string, int, int	—
<code>price_per_night</code>	decimal string	prezzo a notte
<code>nights</code> · <code>subtotal</code>	int, decimal string	subtotale per l'intero soggiorno
<code>services</code>	<code>RoomServiceSchema[]</code>	<code>{id, name}</code>
<code>fits_all_guests</code>	bool	la camera da sola basta per gli ospiti

ROOMCOMBINATIONSCHEMA

`room_ids` (UUID[]) · `rooms_count` (int) · `total_capacity` (int) · `total_price` (decimal string) · `wasted_capacity` (int, posti letto eccedenti)

Porta solo gli identificativi: i dati delle camere sono già in `rooms`.

BOOKINGQUOTERESPONSESCHEMA

check_in · check_out · nights · guest_count · payment_option · lines[] · base_price · discount_amount · total_price · currency · quote_token · quote_expires_at

lines[] è PriceLineSchema : room_id , room_name , unit_price , nights , line_total .

BOOKINGPUBLICSCHEMA — VISTA PER L'OSPITE

Campo	Tipo	Note
code	string	codice leggibile, es. BB-2026-A7K3QX
status	enum	vedi §6
check_in · check_out · nights	date, date, int	—
guest_count	int	—
guest_firstname · guest_lastname · guest_email	string	snapshot anagrafico
rooms	BookingRoomItemSchema[]	—
base_price · discount_amount · total_price · currency	decimal string	—
payment_option · payment_status	enum	—
hold_expires_at	datetime null	scadenza del blocco temporaneo
cancellation_deadline	datetime null	termine di cancellazione gratuita
confirmed_at	datetime null	—

Non contiene id , user_id , version , admin_notes , campi di audit né riferimenti Stripe. C'è un test che lo verifica.

BOOKINGROOMITEMSCHEMA

room_id · room_name · room_number · check_in · check_out · nights · unit_price · line_total

unit_price è il prezzo congelato al momento della prenotazione: una modifica successiva al listino non lo altera.

BOOKINGCREATEDSCHEMA

Campo	Tipo	Note
booking	BookingPublicSchema	—
confirmation_token	string null	solo se EMAIL_ENABLED=false — vedi §3.6

BOOKINGSCHEMA — VISTA AMMINISTRATIVA (STEP E)

Tutti i campi di `BookingPublicSchema`, più: `id`, `source_channel`, `user_id`, `guest_phone`, `payment_method`, `cancelled_at`, `cancellation_reason`, `admin_notes`, `created_at`, `updated_at`, `created_by`, `last_updated_by`, `status_history[]`.

BOOKINGLISTITEMSCHEMA E PAGINATEDBOOKINGSSHEMA (STEP E)

Riga di elenco: `id`, `code`, `status`, `check_in`, `check_out`, `guest_lastname`, `guest_email`, `rooms_count`, `total_price`, `payment_status`. Contenitore: `items[]`, `total`, `page`, `page_size`, `pages`.

6. Enumerazioni

BookingStatus

Valore	Significato	Slot occupato
PENDING_CONFIRMATION	In attesa di conferma via email	sì, finché il blocco è valido
PENDING_PAYMENT	In attesa del pagamento online	sì, finché il blocco è valido
CONFIRMED	Confermata	sì
CHECKED_IN	Ospite arrivato	sì
COMPLETED	Soggiorno concluso	sì
CANCELLED	Annullata (<i>terminale</i>)	no
EXPIRED	Blocco scaduto senza conferma (<i>terminale</i>)	no
NO_SHOW	Ospite non presentato (<i>terminale</i>)	sì

Transizioni ammesse

Da	A
PENDING_CONFIRMATION	CONFIRMED · EXPIRED · CANCELLED
PENDING_PAYMENT	CONFIRMED · EXPIRED · CANCELLED
CONFIRMED	CHECKED_IN · CANCELLED · NO_SHOW
CHECKED_IN	COMPLETED
COMPLETED · CANCELLED · EXPIRED · NO_SHOW	(<i>terminali</i>)

Ogni transizione non prevista produce `409`. Ogni transizione eseguita lascia una riga in `booking_status_history` con attore e motivazione.

Altri enum

Enum	Valori
PaymentOption	PAY_NOW · PAY_ON_ARRIVAL
PaymentStatus	NOT_REQUIRED · PENDING · AUTHORIZED · PAID · FAILED · REFUNDED · PARTIALLY_REFUNDED
PaymentMethod	STRIPE_CARD · CASH_ON_SITE · POS_ON_SITE · BANK_TRANSFER
BookingChannel	PUBLIC_GUEST · PUBLIC_USER · ADMIN_BACKOFFICE
BookingTokenPurpose	CONFIRM_EMAIL · MANAGE · CANCEL
AuditActorType	GUEST · USER · ADMIN · SYSTEM
UserRole	user · admin

7. Copertura dei test

`pytest` dalla root del progetto. Il database di test si crea da solo alla prima esecuzione.

File	Test	Database	Cosa verifica
<code>test_booking_schema.py</code>	38	no	Vincoli dei DTO: date, capienza, XOR utente/ospite, honeypot, normalizzazione codice
<code>test_pricing_service.py</code>	18	no	Sconti, arrotondamento <code>ROUND_HALF_UP</code> , assenza di <code>float</code> , quote token, penali
<code>test_availability_combinations.py</code>	11	no	Minimalità, ordinamento, limiti, euristica su inventari ampi
<code>test_booking_service.py</code>	23	sì	Concorrenza (x10) , ciclo di vita, transizioni, hold scaduto, back-to-back
<code>test_booking_api.py</code>	15	sì	Flusso end-to-end, rate limit, protezioni, autorizzazione
Totale	105		

Il test che conta più di tutti

`test_due_prenotazioni_concorrenti_una_sola_vince` lancia due creazioni identiche con `asyncio.gather` su sessioni distinte e pretende **esattamente un successo e un `RoomNotAvailable`**. È ripetuto 10 volte, perché una race condition che si manifesta una volta su dieci resta una race condition.

L'anti-overbooking non è garantito dal codice applicativo ma da un **exclusion constraint di PostgreSQL**: nessuna sequenza di operazioni concorrenti può produrre una doppia vendita.

8. Checklist di verifica manuale

Da eseguire su Swagger (`/docs`) a sviluppo concluso.

Prerequisiti

- [] Almeno due camere abilitate in `rooms`
- [] Un utente con `role = admin`
- [] `EMAIL_ENABLED=false` (per ricevere il token in risposta)

Flusso ospite — percorso felice

- [] `GET /availability` restituisce le camere libere con `fits_all_guests` corretto
- [] Le combinazioni proposte coprono gli ospiti e non contengono camere superflue
- [] `POST /quote` restituisce un totale coerente e un `quote_token`
- [] `POST /` crea la prenotazione in `PENDING_CONFIRMATION` e restituisce il token
- [] `POST /confirm` porta a `CONFIRMED`, azzera `hold_expires_at`, valorizza `cancellation_deadline`
- [] `POST /lookup` con codice ed email restituisce la prenotazione

Flusso ospite — casi limite

- [] Ripetere `POST /` sullo stesso slot → `409`
- [] Confermare due volte → `409` "già stata confermata"
- [] `POST /` con `website` valorizzato → `422`
- [] `POST /` con `accept_terms: false` → `422`
- [] `POST /quote` con `check_out` precedente a `check_in` → `422`
- [] `POST /quote` con più ospiti della capienza → `422`
- [] Ventun richieste a `/availability` in un minuto → `429` con `Retry-After`
- [] `POST /lookup` con email errata → `404` con messaggio generico

Autorizzazione

- [] `GET /me` senza cookie → `401`
- [] `GET /me` con cookie valido → le proprie prenotazioni
- [] Cancellare la prenotazione di un altro utente → `404`

Integrità dei dati

- [] Modificare il prezzo di una camera dopo una prenotazione → il totale storico resta invariato
- [] Cancellare una camera con prenotazioni → `409 EntityInUse`
- [] Cancellare un utente → le sue prenotazioni sopravvivono con `user_id` a `null`

Da verificare allo Step F

- [] Attesa oltre 15 minuti senza conferma → stato `EXPIRED` e slot riprenotabile
 - [] Con `EMAIL_ENABLED=true`, `confirmation_token` è `null` nella risposta
-

9. Changelog

Data	Step	Modifiche
20/09/2026	D	9 endpoint pubblici e utente · rate limiting · captcha Turnstile · honeypot · <code>Retry-After</code> · <code>BookingCreatedSchema</code> · <code>trusted_proxy_count</code>
20/09/2026	C	Preventivo firmato · token monouso · motore di prenotazione con locking a 3 livelli · disponibilità con combinazioni
19/09/2026	B	21 DTO · 3 repository · <code>AppException</code> con handler unico · correzione 500 → 422 sulle validazioni · <code>EntityInUse</code>
19/09/2026	A	Modelli e macchina a stati · exclusion constraint anti-overbooking · foreign key composita · guest booking

10. Da completare

Step	Contenuto	Impatto su questo documento
E	API amministrative	8 nuovi endpoint sotto <code>/api/v1/admin/bookings</code>
F	Email e sweeper	<code>confirmation_token</code> sparisce dalle risposte; transizione automatica a <code>EXPIRED</code>
G	Pagamenti Stripe	Endpoint webhook e creazione Payment Intent; codici <code>402</code>

Documento generato automaticamente dal sorgente `docs/api/booking_api.md`. Per segnalare una discrepanza fra documento e codice, fa fede il codice.